

ACTIVE DIRECTORY HOME LAB

Phase 2 — Attack Simulation & Detection

Analyst: Bryan Sykes | May 27, 2026 | Active Directory Security Portfolio Project

1. Phase 2 Overview

Project Title	Active Directory Home Lab - Phase 2: Attack Simulation & Detection
Phase	Phase 2 of 2 (builds on Phase 1 environment)
Date	May 27, 2026
Analyst	Bryan Sykes
Attacker Machine	Kali Linux (192.168.1.100) VirtualBox Internal Network
Target Domain Controller	DC01 apexdefense.local (192.168.1.10)
Target Workstation	WS01 Windows 10 Pro, domain-joined (192.168.1.20)
Attacks Executed	LLMNR Poisoning, Kerberoasting, Pass-the-Hash, BloodHound Enumeration
Primary Tools	Responder, Hashcat, Impacket (GetUserSPNs, secretsdump, psexec), bloodhound-python
Classification	CONFIDENTIAL (Home Lab, No Live Systems Targeted)

2. Phase 2 Lab Environment

Phase 2 added two VMs to the Phase 1 Domain Controller environment. A Windows 10 domain-joined workstation and the existing Kali Linux attacker VM reconfigured to the internal lab network. All three VMs communicate over an isolated VirtualBox Internal Network with no internet access.

VM	OS	IP Address	Role
DC01	Windows Server 2022	192.168.1.10	Domain Controller apexdefense.local
WS01	Windows 10 Pro	192.168.1.20	Domain-joined workstation attack target
Kali Linux	Kali 2025.2	192.168.1.100	Attacker machine all attack tools

2.1 WS01 Workstation Setup

Windows 10 Pro was installed on a new VirtualBox VM and joined to the apexdefense.local domain. The following configurations were made to simulate a realistic but vulnerable enterprise workstation:

- Static IP: 192.168.1.20, DNS pointing to DC01 (192.168.1.10)
- Domain joined as WS01.apexdefense.local computer account created in AD Computers container, moved to _DEPARTMENTS > IT OU
- Windows Defender Firewall disabled on all profiles (Domain, Private, Public) intentionally vulnerable for lab purposes
- Network Discovery and File Sharing enabled
- IPv6 disabled on network adapter forced IPv4-only to ensure Responder captures authentication via IPv4
- Logged in as domain user jcarter (APEXDEFENSE\jcarter) simulating a standard employee workstation

2.2 Service Account Setup

A dedicated service account was created on DC01 to serve as the Kerberoasting target simulating a real SQL Server service account commonly found in enterprise environments:

- Account: sqlservice (created via net user command due to domain password policy blocking GUI creation)
- SPN registered: MSSQLSvc/DC01.apexdefense.local:1433 this is what makes the account Kerberoastable
- Password set to a weak password (Password1) for successful crack demonstration

2.3 Troubleshooting — Clock Skew Resolution

A significant troubleshooting challenge was encountered during Phase 2: Kerberos authentication failures due to clock skew between Kali and DC01. Kerberos requires clocks to be within 5 minutes of each other, a security feature designed to prevent replay attacks.

Root cause: DC01's hardware clock was misconfigured with the wrong timezone interpretation. VirtualBox Guest Additions was also continuously overriding manual time changes in Kali by syncing the guest clock to the host. The combination caused a persistent ~5 hour UTC offset between the two machines.

Resolution steps taken:

- Disabled VirtualBox Guest Additions time sync for Kali VM using VBoxManage.exe setextradata on the Windows host
- Disabled systemd-timesyncd NTP service on Kali
- Corrected DC01's daylight saving time setting. DST was disabled causing a 1-hour offset
- Set DC01 Windows Time service to use local CMOS clock as authoritative source
- Added DC01 to Kali's /etc/hosts file to resolve DNS without internet

- Used -dc-host parameter instead of -dc-ip in Impacket to bypass Kerberos DNS resolution issues

Key lesson: Clock synchronization is a foundational requirement for Kerberos-based attacks and defenses. In real enterprise environments, NTP hierarchy and time synchronization health are critical operational concerns. A misconfigured time server can cause widespread authentication failures across the domain.

3. Attack Execution & Analysis

3.1. LLMNR Poisoning

MITRE ATT&CK	T1557.001 Adversary-in-the-Middle: LLMNR/NBT-NS Poisoning and SMB Relay
Severity	HIGH
Tool Used	Responder (pre-installed on Kali)
Screenshots	Screenshots 11-16

What This Attack Does

Link-Local Multicast Name Resolution (LLMNR) and NetBIOS Name Service (NBT-NS) are Windows protocols that broadcast name resolution requests on the local network when DNS fails. When a user attempts to access a resource that DNS cannot resolve (e.g. \\fileshare), Windows broadcasts 'does anyone know where fileshare is?' to the entire subnet. Responder listens for these broadcasts and responds with its own IP address, causing the victim machine to attempt authentication against Kali. The authentication attempt sends an NTLMv2 hash the user's password hash — which can be captured and cracked offline.

Attack Execution

- Started Responder on eth0: `sudo responder -I eth0 -dwv`
- Triggered LLMNR broadcast from WS01 by navigating to \\fileshare in File Explorer
- Responder intercepted the broadcast and sent poisoned responses for LLMNR, MDNS, and NBT-NS
- NTLMv2 hashes captured for APEXDEFENSE\Administrator and jcarter in `/usr/share/responder/logs/`
- Saved jcarter hash to file and ran Hashcat with mode 5600 (NetNTLMv2) against `rockyou.txt`
- First attempt with strong password (Password@123) Hashcat exhausted all 14.3M passwords, no crack
- Changed jcarter password to Password1 (weak, in `rockyou.txt`) and recaptured hash
- Hashcat cracked Password1 in under 3 seconds. Recovered: 1/1 (100%)

Key Findings

- NTLMv2 hashes captured for both Administrator and jcarter without any special privileges, just network position
- Weak passwords in rockyou.txt cracked in seconds, strong passwords not in common wordlists survived
- LLMNR poisoning works because Windows trusts responses from any host on the local network by default
- The attack requires no credentials, no prior access, and no exploitation, just network presence and a listening tool
- IPv6 must be disabled or SMB capture falls back to IPv6 path which Responder cannot intercept on this network configuration

Detection — Windows Event IDs to Monitor

Event ID 4625: Failed logon attempts will appear on DC01 when hash capture triggers authentication. Event ID 4648: Logon using explicit credentials. In more advanced environments, Microsoft Defender for Identity (MDI) detects LLMNR poisoning directly. Network monitoring for unusual LLMNR/NBT-NS response traffic from unexpected hosts is the most reliable detection method.

Prevention Controls

- Disable LLMNR via Group Policy: Computer Configuration > Administrative Templates > Network > DNS Client > Turn off multicast name resolution = Enabled
- Disable NBT-NS via network adapter settings: TCP/IP properties > WINS tab > Disable NetBIOS over TCP/IP
- Enable SMB signing to prevent relay attacks even if hashes are captured
- Enforce strong password policies and MFA — cracking becomes infeasible even with captured hashes
- Network segmentation — limit which hosts can communicate on the same broadcast domain

3.2. Kerberoasting

MITRE ATT&CK	T1558.003 Steal or Forge Kerberos Tickets: Kerberoasting
Severity	HIGH
Tool Used	Impacket-GetUserSPNs (pre-installed on Kali)
Screenshots	Screenshots 17-19

What This Attack Does

Kerberoasting exploits the Kerberos authentication protocol's ticket-granting mechanism. When a domain user requests access to a service with a registered Service Principal Name (SPN), the Domain Controller issues a Kerberos TGS ticket encrypted with the service account's NTLM password hash. Any authenticated domain user can request these tickets — no elevated

privileges required. The ticket is taken offline and cracked to reveal the service account's plaintext password. Service accounts are prime targets because they often have weak passwords, high privileges, and Password never expires enabled.

Attack Execution

- Ran GetUserSPNs to enumerate all accounts with SPNs: `impacket-GetUserSPNs apexdefense.local/jcarter:Password@123 -dc-host DC01.apexdefense.local -request`
- Tool authenticated as low-privilege domain user jcarter and requested all service tickets
- DC01 returned a TGS ticket for sqlservice encrypted with its password hash
- Ticket hash saved to `/home/kali/sqlservice.hash`
- Ran Hashcat with mode 13100 (Kerberos TGS-REP) against `rockyou.txt`
- First attempt with a strong password. Hashcat exhausted wordlist, no crack
- Changed sqlservice password to Password1 and re-ran GetUserSPNs
- Hashcat cracked Password1 in under 2 seconds. Recovered: 1/1 (100%)

Key Findings

- sqlservice account with SPN `MSSQLSvc/DC01.apexdefense.local:1433` was successfully targeted using only jcarter's regular domain credentials
- The DC issued the service ticket without any suspicious authentication. This is legitimate Kerberos behavior being abused
- No alerts were generated on DC01 during the ticket request. Kerberoasting is inherently stealthy
- Weak service account passwords crack in seconds. Password complexity is the primary defense
- Password never expires on service accounts is a critical misconfiguration that enables this attack long-term

Detection — Windows Event IDs to Monitor

Event ID 4769: Kerberos Service Ticket Request. Specifically, requests for RC4 encryption (etype 0x17) are a strong indicator of Kerberoasting since modern Kerberos uses AES encryption by default. High volume of 4769 events from a single source in a short window indicates automated Kerberoasting. Microsoft ATA and Defender for Identity detect this pattern automatically.

Prevention Controls

- Use strong, long, random passwords (25+ characters) for all service accounts makes offline cracking infeasible
- Use Group Managed Service Accounts (gMSA) automatically rotated 240-character passwords that cannot be Kerberoasted
- Audit all accounts with SPNs registered: `Get-ADUser -Filter {ServicePrincipalName -ne '$null'} -Properties ServicePrincipalName`

- Enable AES encryption for Kerberos tickets makes captured tickets significantly harder to crack
- Apply tiered administration model service accounts should have minimum required privileges, not Domain Admin

3.3. Pass-the-Hash

MITRE ATT&CK	T1550.002 Use Alternate Authentication Material: Pass the Hash
Severity	CRITICAL
Tool Used	Impacket-secretsdump and Impacket-psexec (pre-installed on Kali)
Screenshots	Screenshots 20-24

What This Attack Does

Pass-the-Hash exploits Windows NTLM authentication's fundamental design: the protocol never transmits the plaintext password — it uses the NTLM hash directly for authentication. An attacker who captures a valid NTLM hash can present it to authenticate to any system that accepts NTLM authentication, without ever knowing the plaintext password. Combined with secretsdump which dumps all domain NTLM hashes by connecting to the DRSUAPI replication interface an attacker with Domain Admin credentials can extract every hash in the domain and use any of them for lateral movement.

Attack Execution

- Ran secretsdump with Domain Admin credentials to dump all NTLM hashes: `impacket-secretsdump apexdefense.local/Administrator:[password]@192.168.1.10`
- All domain NTLM hashes dumped including Administrator, all users, sqlservice, and computer accounts DC01\$ and WS01\$
- Extracted Administrator NTLM hash: `1cfe3e2ff506a887df7fc15735cedfb9`
- Ran psexec using the hash directly no password needed: `impacket-psexec Administrator@192.168.1.20 -hashes aad3b435b51404eeaad3b435b51404ee:[hash]`
- psexec uploaded service binary to ADMIN\$ share, created and started a Windows service
- Shell returned: `C:\Windows\system32>`
- Ran whoami returned: `nt authority\system` highest privilege level on Windows
- Ran ipconfig confirmed shell executing inside WS01 at 192.168.1.20

Key Findings

- All domain NTLM hashes extracted in a single command every user account in the domain compromised simultaneously
- Administrator hash used to obtain SYSTEM shell on WS01 without knowing the plaintext password

- nt authority\system is the highest privilege level on Windows complete control of WS01 achieved
- The attack traverses the entire domain: one set of Domain Admin credentials = every hash = access to every machine
- Computer account hashes (DC01\$, WS01\$) were also dumped enabling machine-level authentication attacks

Detection — Windows Event IDs to Monitor

Event ID 4624: Successful logon with logon type 3 (network logon) using NTLM authentication.
Event ID 4672: Special privileges assigned to new logon (indicates admin-level access).
Event ID 7045: New service installed (psexec installs a service). Monitoring for ADMIN\$ share access combined with new service creation is a reliable psexec detection pattern. Microsoft Defender for Endpoint detects Pass-the-Hash behavior directly.

Prevention Controls

- Enable Protected Users Security Group for privileged accounts prevents NTLM authentication for group members
- Implement Credential Guard isolates NTLM hashes in a virtualized container inaccessible to secretsdump
- Disable NTLM authentication where possible enforce Kerberos-only authentication
- Apply least privilege limit which accounts have Domain Admin rights to reduce blast radius
- Enable Local Administrator Password Solution (LAPS) unique local admin passwords per machine prevent lateral movement via local admin hashes
- Monitor for DRSUAPI replication requests from non-DC machines secretsdump uses this protocol

3.4. BloodHound Domain Enumeration

MITRE ATT&CK	T1087 - Account Discovery T1069 Permission Groups Discovery
Severity	HIGH
Tool Used	bloodhound-python (pre-installed on Kali at /usr/bin/bloodhound-python)
Screenshots	Screenshots 25-26

What This Attack Does

BloodHound collects comprehensive Active Directory data via LDAP queries using any authenticated domain account, then analyzes the relationships between users, groups, computers, GPOs, and ACLs to identify attack paths to Domain Admin. In a real engagement BloodHound visualizes these paths as a graph showing the exact sequence of hops an attacker must take to escalate from a low-privilege user to full domain compromise. The data collection (SharpHound/bloodhound-python) is difficult to detect because it uses legitimate LDAP queries that resemble normal AD replication traffic.

Attack Execution

- Ran collection with domain user credentials: `bloodhound-python -u jcarter -p Password@123 -d apexdefense.local -dc DC01.apexdefense.local -ns 192.168.1.10 -c all --zip`
- Collection completed in under 1 second 7 JSON files packaged into `20260527204317_bloodhound.zip`
- Extracted zip and analyzed JSON files directly (BloodHound GUI unavailable without Neo4j)
- Identified Domain Admins group members via `groups.json` only Administrator (SID -500)
- Identified computer ACLs via `computers.json` GenericAll, GenericWrite, WriteOwner, WriteDacl rights visible on both DC01 and WS01
- Mapped attack path from jcarter to Domain Admin through group memberships and computer access rights

Key Findings

- Full domain enumeration completed in under 1 second using only regular domain user credentials no elevated privileges required
- 14 users, 56 groups, 4 GPOs, 7 OUs, 2 computers, and 19 containers all mapped in a single command
- Attack path identified: jcarter > IT-Admins group > WS01 admin access > Administrator session > Domain Admins > DC01
- ACL rights (GenericAll, WriteOwner, WriteDacl) on computer objects reveal additional privilege escalation paths beyond group membership
- BloodHound collection is stealthy LDAP queries are indistinguishable from normal AD administrative traffic without specialized monitoring

Detection — Windows Event IDs to Monitor

Event ID 4662: Object access on AD objects (if object access auditing is enabled). High volume of LDAP queries from a workstation in a short time window is the primary detection indicator. Microsoft Defender for Identity has specific detections for BloodHound collection patterns based on the volume and type of LDAP queries. LDAP signing and channel binding enforcement reduces the attack surface for unauthenticated enumeration.

Prevention Controls

- Enable Advanced Audit Policy specifically audit DS Access to log LDAP queries against sensitive AD objects
- Deploy Microsoft Defender for Identity (MDI) has built-in BloodHound detection based on LDAP query patterns
- Implement tiered administration model limits the attack paths BloodHound can find by isolating admin accounts
- Clean up excessive ACL permissions remove unnecessary GenericAll and WriteOwner rights from computer and user objects

- Enable LDAP signing and channel binding reduces unauthenticated LDAP enumeration risk
- Apply least privilege to all accounts fewer high-privilege paths means fewer attack paths to Domain Admin

4. Full Attack Chain Summary

The four attacks executed in Phase 2 represent a complete, realistic AD compromise scenario. When chained together they form a progression from zero domain access to full domain compromise:

Step	Attack	Result	What It Enabled
1	LLMNR Poisoning	NTLMv2 hash captured for jcarter	Credential access plaintext password recovered via offline crack
2	Kerberoasting	TGS hash captured for sqlservice	Service account credential access demonstrates privilege escalation via service accounts
3	Pass-the-Hash	SYSTEM shell on WS01	Full workstation compromise highest privilege level without knowing plaintext password
4	BloodHound	Full domain map + attack paths identified	Attack path visibility shows route from any user to Domain Admin for follow-on attacks

In a real intrusion this chain would progress further: BloodHound attack paths would guide the attacker from the SYSTEM shell on WS01 to Domain Admin on DC01, enabling Golden Ticket creation, persistent backdoor installation, and complete domain persistence. Phase 2 demonstrated the first four steps of that chain.

5. Phase 2 Troubleshooting Log

Issue	Root Cause	Resolution
Responder not capturing hashes — IPv6 errors	WS01 was sending authentication via IPv6 which Responder could not intercept on this network	Disabled IPv6 on WS01 network adapter to force IPv4-only authentication
Hashcat exhausted on strong passwords	Password@123 and SqlSvc@Lab2026! are not in rockyou.txt custom passwords survive common wordlists	Changed passwords to Password1 (known rockyou entry) to demonstrate successful crack for documentation purposes
Kerberos clock skew too great error	5+ hour UTC offset between Kali and DC01 caused by VirtualBox time sync overriding manual	Disabled VBoxManage Guest Additions time sync, corrected DC01

	changes, plus DC01 DST misconfiguration	DST setting, synced clocks manually
Service account creation blocked by password policy	Domain password policy (12 char minimum, complexity required) blocked GUI account creation	Used net user command via CMD to bypass GUI limitation command line accepted the password after Y confirmation
BloodHound GUI unavailable — Neo4j not installed	Neo4j database required for BloodHound GUI was not installed and no internet available to install it	Analyzed BloodHound JSON output files directly using Python json.tool demonstrated equivalent analytical findings
DNS resolution failing for bloodhound-python	Kali's default DNS server couldn't resolve apexdefense.local without internet	Used -ns 192.168.1.10 flag to specify DC01 as nameserver and added DC01 to /etc/hosts

6. Key Technical Lessons

- LLMNR and NBT-NS are enabled by default on all Windows versions and represent one of the most exploited attack vectors in internal network penetration tests. Disabling them via GPO is a quick, high-impact security win that costs nothing.
- Kerberoasting requires zero special privileges; any domain user can request service tickets for any SPN. Service accounts with weak passwords and Password never expires are among the most dangerous misconfigurations in Active Directory.
- Pass-the-Hash demonstrates why hash protection is as important as password protection. An NTLM hash is functionally equivalent to a password for NTLM authentication. Credential Guard and Protected Users are the primary defenses.
- BloodHound collection is nearly undetectable with standard Windows logging. The LDAP queries it generates are indistinguishable from normal AD administrative traffic. Defender for Identity is required for reliable detection.
- Clock synchronization is a foundational Kerberos requirement. A 5-minute tolerance exists specifically to prevent replay attacks; the same security feature that causes operational pain in misconfigured lab environments is protecting production domains from a real attack class.
- Strong passwords are the most practical defense against both LLMNR poisoning and Kerberoasting. Hashes that can't be cracked from a wordlist provide significant protection even when capture is unavoidable.
- The attack chain demonstrates defense-in-depth failures. Each individual control (firewall, strong passwords, network segmentation, privileged account isolation) would have broken the chain at a different point. No single control stops everything — layered defenses are required.

7. Conclusion

Phase 2 of the Active Directory Home Lab successfully executed four of the most common real-world AD attack techniques against the apexdefense.local domain built in Phase 1. LLMNR Poisoning captured and cracked NTLMv2 hashes using only network position. Kerberoasting

extracted and cracked a service account's Kerberos ticket using only regular domain credentials. Pass-the-Hash obtained a SYSTEM shell on WS01 using only a captured hash. BloodHound enumerated the full domain and mapped attack paths to Domain Admin.

Taken together, Phases 1 and 2 demonstrate the complete lifecycle of an Active Directory environment from building and administering a domain to attacking it and understanding the detections and preventions that would stop each technique. This dual perspective administrator and attacker is the foundation of effective Active Directory security.

The troubleshooting log documents six real challenges encountered during Phase 2, including a complex clock synchronization issue that required modifying VirtualBox hypervisor settings from the Windows host. These are not failures, they are the documentation of how security practitioners actually work in lab and production environments, and the diagnostic reasoning applied to resolve them is itself a portfolio skill.

Documentation prepared by: **Bryan Sykes** | May 27, 2026 | Active Directory Home Lab — Phase 2